A SURVEY OF PRACTICE

The Exceptional Individual Contributor

How staff and principal engineers work, and how an early-stage principal AI engineer becomes much better at the job

---

Compiled October 2026 from published practitioner writing, books, and engineering guidance. Sources are listed at the end.

Contents

How to read this survey 3
Glossary 4

PART I. FOUNDATIONS

1. What a staff or principal engineer actually is 10
2. Choosing the work that matters 12
3. Knowing where you are: the three maps 14

PART II. SETTING DIRECTION

4. Technical strategy and vision 16
5. Making and recording decisions 18

PART III. EXECUTION

6. Leading large projects and actually shipping them 20
7. Reliability, incidents, and learning from failure 22

PART IV. INFLUENCE AND PEOPLE

8. Influence without authority and organizational dynamics 24
9. Writing and communication 26
10. Multiplying others: glue work, mentoring, sponsorship 28

PART V. CRAFT

11. Staying technical at a senior level 30
12. Technical judgment and taste 32

PART VI. THE AI ENGINEERING LAYER

13. Evaluation as the central discipline 34
14. Architecture judgment for agentic systems 36
15. Context engineering 38
16. Running AI systems in production 40
17. Working with coding agents 42

PART VII. GROWTH

18. Deliberate growth and keeping up with the field 44
19. A twelve-month plan for an early-stage principal AI engineer 46

Summary table 48
Sources and further reading 50

How to read this survey

This document collects what the best published practitioners say about doing exceptional work as a senior individual contributor, with particular attention to people who hold, or have recently stepped into, a principal role in AI engineering. It draws on books such as Will Larson's *Staff Engineer* and *Crafting Engineering Strategy*, Tanya Reilly's *The Staff Engineer's Path*, Amazon's published Principal Engineering Community tenets, long-form writing by working staff engineers such as Sean Goedecke, and, for the AI-specific chapters, Anthropic's engineering guidance on agents, evaluations, and context, along with the error-analysis practice popularized by Hamel Husain and Shreya Shankar.

The phrase "state of the art" is a little unusual when applied to a career rather than a model, but it fits. The thinking about what senior engineers should do has changed substantially over the past decade. The older view treated a principal engineer as the person who writes the hardest code. The current view, which most of the sources agree on, treats the role as one of owning outcomes across a scope larger than any single team, using technical depth as the source of credibility and judgment rather than as the whole job. For AI engineers there is a second shift on top of the first: the systems being built are probabilistic, so the craft has moved toward measurement, evaluation, and careful control of what the model sees.

Each chapter follows the same structure so that it can be read on its own. It begins with a short summary, then explains the problem the practice addresses, describes the idea in plain terms, gives a concrete example, lays out the alternatives people commonly use, and closes with a view on what works best and why. The judgments about "what works best" reflect the weight of the sources rather than a single author's preference, and where the sources disagree, the disagreement is noted.

The glossary that follows defines the terms used throughout. Ordinary engineering vocabulary is left out; the glossary covers terms that carry a specific meaning in this literature or that are easy to misunderstand.

Glossary

Terms are grouped by theme and listed roughly in the order in which they appear in the survey.

Roles and career

Individual contributor (IC)

An engineer whose career progresses without managing people. Senior IC levels typically run senior, staff, senior staff, principal, and distinguished, though names and numbers vary by company.

Staff engineer

The first IC level above senior at most companies. The defining expectation is impact beyond a single team, usually across a group of teams or a product area.

Principal engineer

An IC level above staff whose scope is typically an entire organization or a company-wide technical domain. Principals are expected to shape multi-year direction and to be trusted on decisions that are expensive to reverse.

Staff-plus

A shorthand for all IC levels at or above staff.

Scope

The size of the problem space for which you are accountable: a component, a team's systems, a group of teams, an organization, or the company. Promotion at senior IC levels is largely a recognition that someone already operates at a larger scope.

Leverage

The ratio between the impact of a piece of work and the time it takes. Senior ICs are expected to find work where an hour of their time produces far more than an hour of someone else's.

Archetypes (Tech Lead, Architect, Solver, Right Hand)

Will Larson's four common shapes of staff-plus work. A Tech Lead guides the approach and execution of one or a few teams. An Architect owns the direction and quality of a technical area across teams. A Solver goes deep on a hard, specific problem and then moves on. A Right Hand extends a senior leader's attention and acts on their behalf across the organization.

DRI (directly responsible individual)

The single person accountable for a project's outcome. The term comes from Apple and is widely used. Having a DRI does not mean that person does all the work; it means there is no ambiguity about who is answerable.

Promotion packet and brag document

A promotion packet is the written case a manager or committee uses to decide on a promotion. A brag document, a term popularized by Julia Evans, is a running personal record of your work and its impact, kept so that the packet can be written accurately.

Choosing work and influence

Snacking

Hunter Walk's term, adopted by Larson, for choosing easy, low-impact work because it feels productive. A useful test is whether a capable engineer two levels below you could do it just as well.

Preening

Larson's term for work that is highly visible but has little real impact. It can be rewarded in the short term because many organizations confuse visibility with impact.

Chasing ghosts

Larson's term for pushing changes based on an earlier company's problems without first understanding the current situation, which is a common failure for newly hired senior people.

Glue work

Tanya Reilly's term for the less visible work that makes a team effective, such as coordinating across teams, onboarding, improving process, reviewing designs, and noticing dropped responsibilities. It is essential at senior levels and can hurt the careers of people who do it before it is recognized as part of their role.

Mentorship and sponsorship

Mentorship is giving someone advice and feedback. Sponsorship is spending your own reputation and access to create opportunities for someone, for example by recommending them for a high-visibility project or naming their work in a room they are not in.

The three maps

Tanya Reilly's framing for situational awareness. The locator map shows where your team sits in the larger organization. The topographic map shows the terrain: how decisions are really made, where the power and friction are. The treasure map shows where things should be heading.

Pre-wiring (also nemawashi)

Talking through a proposal one-on-one with the key stakeholders before the formal meeting in which it will be decided, so that concerns are heard and addressed early. Nemawashi is the Japanese term from the Toyota tradition.

BLUF (bottom line up front)

A writing convention, originally military, in which the conclusion or request appears in the first sentence or two, followed by supporting detail.

Strategy and decisions

Technical vision

A description of a desired future state of the systems, typically two to five years out, written concretely enough that teams can recognize whether a decision moves toward it.

Technical strategy

A plan for getting from the present to a desired state given real constraints. Unlike a vision, a strategy makes trade-offs explicit and says what will not be done.

Strategy kernel (diagnosis, guiding policy, coherent actions)

Richard Rumelt's structure for good strategy, from *Good Strategy Bad Strategy*. The diagnosis explains what is actually going on and what makes it hard. The guiding policy is the overall approach chosen to deal with the diagnosis. The coherent actions are the concrete steps that carry out the policy and reinforce each other.

Design document

A written proposal for how a system or change will be built, including context, goals, non-goals, the chosen design, alternatives considered, and risks. Google's engineering culture made it widespread.

RFC (request for comments)

A document circulated to invite feedback before a decision is made. In many companies it is used interchangeably with design document; some reserve it for cross-team or organization-wide proposals.

ADR (architecture decision record)

A short, permanent record of one significant decision: the context, the decision, and its consequences. The format was proposed by Michael Nygard. ADRs accumulate into a decision log that explains why the system looks the way it does.

One-way and two-way doors

Jeff Bezos's distinction between decisions that are hard or impossible to reverse (one-way) and those that can be cheaply undone (two-way). The first kind deserves deliberation; the second deserves speed.

Disagree and commit

The practice of voicing disagreement clearly during a decision and then fully supporting the decision once made, rather than reopening it or undermining it quietly.

Execution and reliability

Critical path

The chain of dependent tasks that determines the earliest date a project can finish. Work on the critical path delays the whole project if it slips; other work does not.

Premortem

Gary Klein's technique of imagining, before a project starts, that it has failed, and asking each participant to explain why. It surfaces risks people would not otherwise voice.

Feature flag

A runtime switch that turns a code path on or off without a deployment, used to release changes gradually, to limited audiences, or with a quick way back.

Blameless postmortem

A written review of an incident that focuses on how the system and process allowed the failure, rather than on who made a mistake. The practice is described in detail in Google's *Site Reliability Engineering* book.

SLO (service level objective) and error budget

An SLO is a target for a measurable property of a service, such as ninety-nine point nine percent of requests succeeding over thirty days. The error budget is the amount of failure the SLO allows, which can be spent on risky changes; when it runs out, reliability work takes priority.

Craft and judgment

Technical debt

The future cost created by choosing an expedient solution now. It is not always bad; like financial debt, it can be a deliberate and sensible choice if the interest is understood.

Hyrum's Law

The observation, named after Hyrum Wright at Google, that with enough users every observable behavior of a system will be depended on by someone, whether or not it is documented. It explains why changing widely used systems is hard.

Innovation tokens

Dan McKinley's idea, from the essay "Choose Boring Technology," that a team can afford only a small number of novel technologies at once, so each one should be spent where it creates real advantage.

AI engineering

Eval (evaluation)

A test of an AI system: an input, a way of running the system on it, and logic that grades the result. Evals play the role for probabilistic systems that unit and integration tests play for deterministic ones.

Task, trial, grader

In Anthropic's terminology, a task is one test case with defined inputs and success criteria, a trial is one attempt at a task (several are run because outputs vary), and a grader is the logic that scores some aspect of a trial.

Transcript, trace, trajectory

The complete record of one run of an AI system, including every model call, tool call, intermediate output, and final answer. The three words are used interchangeably.

Error analysis

The practice of reading many real traces, writing down what went wrong in each, grouping those notes into categories of failure, and counting them to find which failures matter most. It is the recommended starting point for deciding which evals to build.

Open coding and axial coding

Two steps borrowed from qualitative research and applied to error analysis. Open coding means writing free-form notes about each failure without a predefined scheme. Axial coding means grouping those notes into a smaller number of failure categories.

LLM-as-judge

Using a language model as a grader, typically with a rubric, for qualities that code cannot check easily, such as tone, faithfulness to a source, or whether an answer addresses the question. A judge must be validated against human labels before it is trusted.

Capability eval and regression eval

A capability eval measures something the system cannot yet do reliably and is expected to start with a low pass rate. A regression eval checks behavior that already works and should stay near a hundred percent, catching backsliding.

pass@k and pass^k

Two ways of summarizing repeated trials. pass@k is the probability that at least one of k attempts succeeds, which suits settings where a single good answer is enough. pass^k is the probability that all k attempts succeed, which suits user-facing systems that must be consistent.

Golden dataset

A curated set of inputs with known-good expected outputs or labels, used as a stable reference for evaluation.

Eval saturation

The state in which a system passes nearly every task in an eval suite, so the suite no longer distinguishes better versions from worse ones.

Agentic system, workflow, agent

Anthropic's umbrella term and its two subtypes. In a workflow, the steps and their order are fixed in code and the model fills in each step. In an agent, the model decides which steps to take and which tools to call, in a loop, until it judges the task done.

Agent harness (scaffold)

The code around a model that turns it into an agent: the loop, the tool definitions, memory, error handling, and the logic that assembles the context. When people evaluate "an agent," they are evaluating the model and the harness together.

Tool calling

The mechanism by which a model requests that the surrounding program run a function, such as a search or a database query, and return the result into the conversation.

ACI (agent-computer interface)

Anthropic's term for the design of the tools, parameters, and documentation an agent uses, by analogy with human-computer interface design. Good ACI design removes whole classes of agent error.

Context window

The finite amount of text, measured in tokens, that a model can consider at once. Everything the model knows about the current task must fit in it.

Context engineering

The discipline of deciding what goes into the context window at each step: instructions, retrieved documents, tool results, history, and memory. It extends prompt engineering from a single static prompt to the whole lifecycle of an agent's context.

Compaction

Summarizing a long conversation or agent history when it approaches the context limit and continuing from the summary, so that a long task can proceed without losing essential state.

RAG (retrieval-augmented generation)

Fetching relevant documents from an external store at request time and placing them in the context so that the model can ground its answer in them.

Guardrails

Checks placed before or after model calls that block, modify, or escalate inputs and outputs that violate policy, for example filtering personal data or refusing actions outside an allowed set.

Observability (for AI systems)

The ability to see, after the fact, exactly what an AI system did on a given request: its inputs, the full trace of model and tool calls, latencies, costs, and outcomes.

Coding agent

An AI agent that reads and edits a codebase, runs commands and tests, and iterates toward a goal, such as Claude Code or OpenAI Codex.

Vibe coding and agentic engineering

Vibe coding, a term coined by Andrej Karpathy, describes building software by prompting an AI and accepting its output without close review. Agentic engineering, the term Simon Willison now favors over his earlier "vibe engineering," describes the disciplined professional counterpart: using coding agents with specifications, tests, review, and engineering judgment to produce production-quality software.

PART I. FOUNDATIONS

1. What a staff or principal engineer actually is

> *A senior IC is defined less by the difficulty of the code they write than by the size of the problem they own and the way they multiply the work of others. The archetypes describe common shapes of the role, but the role itself is better understood as accountability for technical outcomes across a large scope.*

The problem it addresses

When an engineer is promoted past senior, the expectations change, but they are rarely written down clearly. Many new staff and principal engineers keep doing what made them successful as senior engineers, namely solving hard problems with their own hands, and are then surprised to find that this no longer counts as performing at level. Others overcorrect and stop building anything, spending their days in meetings and producing documents that nobody acts on. Both failures come from the same source: not having a working definition of what the job is for.

The idea, explained simply

The most useful definition in the literature combines two ideas. The first is scope. A senior engineer is accountable for the quality of a team's systems; a staff engineer for a group of teams or a product area; a principal engineer for an organization or a domain that spans the company. The second is leverage. At higher levels, your available time does not grow, but the expected impact does, so you must find work where your contribution changes the trajectory of many people's work rather than adding one more person's worth of output.

Larson's four archetypes are a way of describing how people with this kind of scope actually spend their days. The Tech Lead keeps one or a few teams aligned and moving. The Architect owns the direction of a technical area across teams. The Solver is pointed at the hardest current problem and stays until it is resolved. The Right Hand acts as an extension of a senior leader, carrying their priorities into places they cannot personally reach. Larson is explicit that a single person may combine several archetypes or move between them over time, and later commentators have argued that an archetype is best thought of as a season of a career rather than an identity.

Amazon's Principal Engineering Community tenets add a useful counterweight. Among them is the expectation that a principal remain an exemplary practitioner, meaning close enough to the real technical details to make sound calls, and that they respect what came before, meaning they assume earlier decisions had reasons before deciding to change them. The combination matters: scope and leverage define the job, but technical depth is what makes a principal's judgment worth trusting.

EXAMPLE

Consider an engineer recently promoted to principal in a company building LLM-based assistants for several products. As a senior engineer, they built the retrieval pipeline for one assistant and were known as the person who could debug anything. In the new role, three product teams each

run their own retrieval stack, their own prompt management, and their own informal testing. Nobody can say whether a model upgrade makes the products better or worse.

Acting as a Solver, they could rewrite one team's pipeline to be excellent. Acting at principal scope, they instead notice that the shared missing piece is evaluation: none of the teams can measure quality, so none of them can improve deliberately. They build a small shared evaluation harness with one team, prove it catches a real regression during a model upgrade, and then help the other two teams adopt it. Their personal code output is lower than before, but every team now improves faster, and the organization can make model decisions on evidence. That is what operating at scope looks like.

Alternatives people use

The most common alternative is to treat staff and principal as "senior, but more so," measured by personal output and technical brilliance. This works in small companies and in research-heavy environments where one person's breakthrough is the product, but in most organizations it caps impact at what one person can build. A second alternative is the "manager without reports" model, in which the senior IC mostly coordinates and stops doing technical work. It produces short-term alignment but erodes the technical credibility on which the role depends. A third is to pick an archetype and treat it as a fixed job description, which can be clarifying but often leads to doing the archetype's work even when the organization needs something else.

What works best

The sources converge on defining the role by scope and leverage, with technical depth as a non-negotiable foundation. In practice, this means agreeing explicitly with your manager on the scope you are accountable for, identifying which archetype the organization needs from you at this moment, and revisiting that agreement every few months. It also means keeping some real technical work in your week, not as a hobby but because it keeps your judgment calibrated. For an early-stage principal AI engineer, the most common gap is not technical ability but the shift from "I solve the hardest problem" to "I make sure the organization solves the right problems."

2. Choosing the work that matters

> *The most consequential skill of a senior IC is selecting what to work on. Larson's guidance is to avoid easy work and merely visible work, to understand the current situation before changing it, and to concentrate on existential problems, neglected but important areas, growing others, unblocking stalled work, finishing projects, and the work only you are positioned to do.*

The problem it addresses

Senior engineers have more freedom to set their own agenda than anyone else in engineering. That freedom is the source of most of their impact and most of their failure. Without a deliberate way of choosing, the calendar fills with requests, reviews, and interesting side problems, and a year later it is hard to point to anything that changed because of you.

The idea, explained simply

Larson's essay "Work on What Matters" names three traps. Snacking is doing easy, satisfying work with little impact; the test is whether a capable engineer two levels below you could do it as well. Preening is doing work that is very visible but changes little, which some organizations reward because they mistake visibility for impact. Chasing ghosts is arriving with a diagnosis from a previous job and pushing changes before understanding why the current system looks the way it does.

He then describes where attention is best spent. If the company faces an existential problem, such as running out of money or a severe reliability crisis, that comes first. Otherwise, the best opportunities are often in areas that are important but neglected, because existential problems attract crowds while neglected areas give you room to make a real difference. Growing the team through onboarding, mentoring, and coaching is consistently undervalued and consistently high-leverage. Editing, meaning small interventions such as a short conversation or a quick fix that unblock someone else's stalled work, is cheap for a senior person and expensive for anyone else. Finishing projects matters because value only arrives when work is complete, and senior engineers are often best placed to rescope a stalled effort into something that can ship. Finally, there is the work only you can do, found at the intersection of your particular strengths and what you care about.

EXAMPLE

A principal AI engineer has three candidate projects for the next quarter. The first is building an elegant prompt-templating library, which they would enjoy and which several engineers have asked for. The second is presenting the team's agent architecture at an internal all-hands. The third is investigating why a customer-facing agent's task success rate has drifted down by several points over two months, a problem nobody owns because it sits between the model platform team and the product team.

The templating library is snacking: useful, but a mid-level engineer could build it, and its effect on outcomes is small. The presentation is mostly preening: worth doing briefly, but not as a primary goal. The drift investigation is neglected, important, and well suited to someone with access to both teams and enough depth to read traces. The engineer spends most of the quarter on it, finds that a retrieval index has gone stale and that a recent tool-description change confused the model,

fixes both, and adds a regression eval and an index-freshness alert. They then hand the templating idea to a mid-level engineer as a growth project, which is itself high-leverage.

Alternatives people use

Many senior engineers let their work be chosen for them by the queue of incoming requests. This feels responsive and keeps stakeholders happy but rarely produces large outcomes. Others pursue whatever is most technically interesting, which can produce excellent work in areas nobody needed. Some align purely with what their manager or director says is important; this is safer, but it outsources the judgment that is supposed to be the senior IC's contribution, and it misses problems leadership cannot see.

What works best

The sources favor a deliberate, periodically reviewed portfolio. A practical form is to write down, once a quarter, the two or three things you intend to change, explain why each matters to the business, and check them with your manager and a peer. Track roughly how your time divides between that portfolio and everything else, accepting that some reactive work is part of the job. Before starting anything large, spend time understanding the current state and its history so that you are not chasing ghosts. Treat growing other engineers and finishing stalled work as first-class items in the portfolio rather than as things done in leftover time.

3. Knowing where you are: the three maps

> *Before you can choose good work or influence decisions, you need an accurate picture of your organization: where your area fits, how decisions actually get made, and where things should be heading. Tanya Reilly calls these the locator map, the topographic map, and the treasure map.*

The problem it addresses

Engineers who are brilliant inside their own system often have a vague sense of what happens outside it. They do not know which other teams depend on them, who really makes the decisions that affect them, or what leadership is worried about this year. As a result, they propose things at the wrong time, to the wrong people, in the wrong language, and good ideas die for reasons that have nothing to do with their technical merit.

The idea, explained simply

Reilly's first map, the locator map, answers the question "where am I?" It places your team and systems in the larger organization: who you depend on, who depends on you, what the business is trying to achieve, and how your work connects to it. The second, the topographic map, describes the terrain: the formal and informal decision-making processes, the people whose opinions carry weight, the places where projects reliably get stuck, the history of past initiatives and why they succeeded or failed. The third, the treasure map, describes the destination: where the organization should be in a few years and the rough path to get there. Reilly's point is that most staff-level failures come from operating with one or more of these maps missing or wrong, and that building them is real work that has to be done on purpose.

EXAMPLE

An engineer joins a company as a principal AI engineer and is asked to "improve agent quality." In their first six weeks, they deliberately build the three maps. For the locator map, they draw a diagram of every team touching the agent stack, including data, platform, security, and the product teams, and note which business metrics each cares about. For the topographic map, they hold one-on-ones with about fifteen people and ask the same questions: what is going well, what is frustrating, what was tried before and why it stopped. They learn that a previous attempt to centralize prompt management failed because product teams felt it slowed them down, and that the security team quietly holds a veto on anything involving customer data. For the treasure map, they draft a one-page picture of where agent quality work could be in two years and test it with several people.

When they later propose a shared evaluation platform, they avoid the mistakes of the earlier centralization effort, involve security from the beginning, and frame the proposal in terms of the metrics each product team already tracks.

Alternatives people use

A common alternative is to rely on the org chart, which shows formal reporting lines but almost nothing about where decisions are really made. Another is to learn the terrain by trial and error, discovering

hidden stakeholders only after a proposal has been blocked. Some engineers rely entirely on their manager to translate organizational context, which works when the manager is excellent and fails silently otherwise.

What works best

Deliberate, structured listening early in a role, and periodically afterwards, is consistently recommended. Concretely, this means a round of conversations with a consistent set of questions, a written summary of what you learned (kept private if it includes candid observations about people), and an explicit attempt to understand the history of past initiatives before proposing new ones. Many experienced principals repeat a lighter version of this exercise every six to twelve months because organizations change faster than people's mental models of them.

PART II. SETTING DIRECTION

4. Technical strategy and vision

> *A good technical strategy starts with an honest diagnosis of the current situation, chooses an approach that addresses it, and commits to specific actions, including saying what will not be done. Larson recommends building strategy bottom-up from real decisions rather than writing it top-down from aspirations.*

The problem it addresses

Without shared direction, every team makes locally reasonable choices that add up to an incoherent whole: five ways to call models, three vector databases, incompatible evaluation practices, and no way to make a change across all of them. Leadership often senses this and asks senior engineers for "a strategy," but most strategy documents fail. Some are lists of goals with no plan. Some are aspirational visions disconnected from what teams can actually do. Some are mandates nobody follows.

The idea, explained simply

Rumelt's strategy kernel is the most widely used structure. A diagnosis describes what is really happening and why it is hard, in terms specific enough that someone could disagree with it. A guiding policy describes the overall approach chosen in response, which necessarily rules out other approaches. Coherent actions are the concrete steps that carry out the policy and support one another. Larson's *Crafting Engineering Strategy* adapts this for engineering, describing strategy as an honest diagnosis combined with a practical approach, and breaks the work into exploring the landscape, diagnosing, refining options through methods such as systems modeling and small experiments, setting policy, and then operating the strategy through reviews, exceptions, and updates.

A technical vision is a related but distinct artifact. It describes a desired future state concretely enough that people can tell whether their decisions move toward it. Reilly and Larson both note that vision without strategy floats free of reality, while strategy without vision tends to optimize the present without a destination.

Larson also offers a practical method for arriving at strategy: write several design documents for real decisions, look at what they have in common, and generalize those common choices into a strategy. Then, once several strategies exist, extrapolate them into a vision. This keeps strategy grounded in actual decisions rather than in abstractions.

EXAMPLE

A principal AI engineer writes a strategy for how the organization uses language models. The diagnosis says: teams call four providers through different client code, nobody can compare quality across models because there is no shared evaluation, costs are rising faster than usage because large models are used for simple steps, and model upgrades are risky because behavior changes are discovered by customers.

The guiding policy says: all model access goes through a single internal gateway that handles routing, logging, and cost attribution; any production prompt or agent must have an eval suite before it can be changed; and model selection is decided per step based on measured quality and cost, not by default. It also says explicitly that the organization will not build its own model hosting this year, and that teams may keep their own orchestration code as long as they use the gateway.

The coherent actions include migrating the two highest-traffic products to the gateway in the first quarter, publishing a minimal eval template, and introducing a monthly model review where upgrade decisions are made on evidence. Each action reinforces the others: the gateway makes traces available, traces feed evals, and evals make model decisions safe.

Alternatives people use

The top-down vision document, written by one senior person and announced, is common and usually ignored, because teams were not involved and the document does not engage with their constraints. The technology radar or approved-tools list is useful for narrow questions but is not a strategy because it does not explain why or set direction. Mandates from leadership achieve compliance but rarely commitment and tend to produce exceptions that hollow them out. Finally, many organizations have no explicit strategy at all and rely on informal consensus among senior engineers, which works until those engineers leave or disagree.

What works best

The weight of the sources favors strategies that are diagnosis-first, written, specific about trade-offs, and built from real decisions. The test of a good strategy is whether it helps someone make a decision they would otherwise have been unsure about, and whether it says no to something. Strategies also need an operating mechanism, such as a regular review where exceptions are discussed and the document is updated, or they decay. For an early-stage principal, the practical advice is to start small: write a strategy for one well-defined area where you already have credibility, make sure it is adopted, and only then attempt broader ones.

5. Making and recording decisions

> *Senior engineers spend much of their influence on decisions. The best practice is to match the decision process to how reversible the decision is, to write down significant decisions with their context and alternatives, and to make it clear who decides.*

The problem it addresses

Engineering organizations suffer from two opposite decision failures. Some decide too slowly, debating reversible choices for weeks while work stalls. Others decide too quickly, making expensive, irreversible commitments in a meeting without understanding the consequences. Both are made worse when nobody writes decisions down, so that a year later nobody remembers why something was done and the same debate repeats.

The idea, explained simply

The first tool is the distinction between one-way and two-way doors. A two-way door, such as trying a new prompt structure behind a feature flag, can be walked back through cheaply, so it should be decided quickly by the people closest to it. A one-way door, such as choosing a data model that external customers will integrate against, deserves careful analysis, broad input, and a senior decision-maker.

The second tool is the written proposal. A design document or RFC describes the problem, goals and non-goals, the proposed approach, the alternatives considered and why they were rejected, and the risks. Writing forces clarity, allows people to comment asynchronously, and leaves a record. The third tool is the ADR, a short document capturing a single decision and its consequences. Where design documents can be long and describe a whole system, ADRs are small and accumulate into a log that new engineers can read to understand why things are as they are.

The fourth tool is clarity about who decides. Frameworks such as RACI or a simple "decider" field on a document prevent the common situation in which everyone has a voice but nobody has the authority to close the discussion. Once a decision is made, disagree and commit asks those who argued against it to support it fully, while preserving the option to revisit it if new evidence appears.

EXAMPLE

A team must decide whether its customer support agent should be built as a fixed workflow or a more autonomous agent, and which vector store to use for its knowledge base. The principal engineer treats these differently. The vector store choice is fairly reversible because retrieval sits behind an internal interface, so they ask the team to run a one-week comparison on a sample of real queries and let the tech lead decide. The architecture choice affects how the product is evaluated, how safety reviews work, and what customers will come to expect, so it is closer to a one-way door. They ask for a short design document comparing the two options against the team's eval results, review it with security and the product manager, and record the outcome as an ADR stating that the agent will start as a workflow with a single autonomous sub-step for search, with explicit criteria for when more autonomy would be reconsidered.

Alternatives people use

Many teams decide in meetings and record nothing, which is fast but loses context and allows decisions to be quietly reopened. Some organizations require heavyweight architecture review boards for everything, which ensures scrutiny but slows reversible work to a crawl and concentrates power in a small group. Others rely on consensus, which feels inclusive but often means that the most persistent person wins or that nothing is decided.

What works best

The practice that holds up best is proportionality: light, fast, local decisions for reversible choices, and written, reviewed, explicitly owned decisions for irreversible ones. Keep design documents short enough to be read, include alternatives honestly rather than as straw men, and name the decider. Maintain an ADR log for any system that will outlive its current team. As a principal, one of the most valuable things you can do is to recognize which kind of door a decision is, since teams often misjudge this, and to say so early.

PART III. EXECUTION

6. Leading large projects and actually shipping them

> *Large, cross-team projects do not ship on their own. Someone must own the outcome end to end, keep leadership's trust through regular honest communication, remove blockers before they stall the work, and treat risk as something to surface early rather than hide.*

The problem it addresses

Many projects led by talented engineers stall in the last twenty percent. The code may be written, but a dependency on another team is unresolved, a legal review has not been requested, leadership is not sure whether the project is still a priority, or the launch criteria were never agreed. Reilly devotes a whole part of her book to this, including a chapter on why projects stop, and Goedecke argues that shipping is fundamentally a social achievement: a project counts as shipped when the people who sponsored it agree that it has shipped and that it achieved its purpose.

The idea, explained simply

The first principle is single ownership. One person, the DRI or technical lead, holds an end-to-end understanding of the system and of why the project matters to the business. The second principle is that leadership trust is the scarce resource. Leaders have limited technical context and rely on the lead's estimates and judgment, so the lead must keep them informed through regular updates, raise delays early with explanations, and present problems together with plans. Goedecke suggests keeping a meaningful share of your own time free from implementation, increasing sharply near launch, so that you can respond when problems appear.

The third principle is to reduce risk early. Practices include making the scariest changes first, deploying early behind feature flags so real users and stakeholders can try the work, repeatedly asking "could we ship right now, and if not, what is stopping us?", and holding a premortem in which the team imagines the project has failed and explains why. Reilly adds that projects stop for identifiable reasons, such as being blocked, being lost, having no clear finish line, or being declared done before they deliver value, and that a senior lead's job is to notice which of these is happening and address it directly.

EXAMPLE

A principal engineer leads the migration of three products onto a shared agent platform. Early on, they write a one-page project brief stating the goal in business terms, the definition of done (all three products serving production traffic through the platform with equal or better eval scores), and the main risks. They run a premortem, which surfaces two concerns nobody had raised: that the security team had not reviewed the platform's tool permission model, and that one product's latency budget was tighter than the platform could meet.

They address both in the first month rather than the last. They post a short written update every week to a channel that leadership reads, using the same format each time: progress, risks, decisions needed. When the latency issue turns out to require a two-week delay for one product,

they raise it as soon as it is known, with a plan to ship the other two on schedule. Each product moves behind a feature flag with a small percentage of traffic first, and the eval dashboard is shared with product managers so the definition of done is visible to everyone.

Alternatives people use

Some organizations rely on project managers to drive delivery while engineers focus on code. This can work, but without an engineer who understands the system end to end, technical risks surface late. Others use heavyweight project management with detailed plans and Gantt charts, which provides an illusion of control but adapts poorly to the discovery inherent in AI projects. At the other extreme, many teams have no explicit project leadership and rely on everyone doing their part, which works for small projects and fails for cross-team ones.

What works best

A single, technically deep owner with explicit responsibility for the outcome, a written definition of done agreed with sponsors, a regular and predictable update cadence, early surfacing of risks, and incremental delivery that puts real work in front of real users as soon as possible. For AI projects in particular, the definition of done should be expressed in terms of measured quality on an agreed eval set, because "the agent works" is too vague to know when you have arrived.

7. Reliability, incidents, and learning from failure

> *Senior engineers set the culture for how failure is handled. The established practice, from Google's SRE tradition and elsewhere, is to define reliability targets explicitly, respond to incidents in a structured way, and learn from them through blameless reviews that change the system rather than blaming individuals.*

The problem it addresses

Every production system fails. The question is whether the organization learns from failures or merely recovers from them. Without explicit reliability targets, teams argue indefinitely about whether a system is "reliable enough." Without structured incident response, outages last longer and are more chaotic. Without blameless review, people hide mistakes and the same failure recurs. AI systems add new failure modes, such as quality degrading silently without any error being thrown, that traditional monitoring does not catch.

The idea, explained simply

An SLO sets a measurable target for what users experience, such as the share of requests that succeed or complete within a time limit. The error budget is the amount of failure that target allows. When the budget is healthy, teams can take risks; when it is exhausted, they prioritize reliability. This turns an emotional argument into a quantitative one.

During an incident, a clear structure helps: one incident commander who coordinates, separate people for communication and for investigation, and a shared timeline. After the incident, a blameless postmortem asks how the system and process made the failure possible and what changes would prevent it or reduce its impact. The word blameless does not mean nobody is accountable; it means the analysis assumes people acted reasonably given what they knew, and looks for the conditions that misled them.

For AI systems, reliability must include quality. A model provider change, a prompt edit, or a shift in user behavior can make an agent worse without causing a single error. Mature teams therefore define quality SLOs, such as a minimum score on a continuously sampled online evaluation, and treat sustained drops as incidents.

EXAMPLE

A document-question-answering assistant starts producing confident but wrong answers for a subset of customers. No errors appear in the logs, so the problem is reported by a customer after several days. The principal engineer runs a blameless review. It finds that a data pipeline change had stopped indexing a category of documents, so retrieval returned nothing relevant and the model answered from general knowledge. Nobody had made an unreasonable choice: the pipeline team had no way to know the assistant depended on that category, and the assistant had no check for empty retrieval.

The resulting actions are systemic. The assistant now detects when retrieval returns low-relevance results and declines to answer or says it is uncertain. An online grader samples production traffic daily and alerts when groundedness drops below a threshold. The pipeline publishes a freshness

metric per document category, and the dependency is documented. The review document is shared widely so that other teams can check whether they have the same exposure.

Alternatives people use

Some organizations aim for maximum reliability without explicit targets, which leads either to endless investment in reliability or to arbitrary decisions about when to stop. Others treat incidents as individual failures and look for someone to blame, which works against honest reporting. Many AI teams rely on user complaints as their quality monitoring, which means problems are discovered late and by the people least willing to tolerate them.

What works best

Explicit SLOs that include quality measures for AI systems, a lightweight but consistent incident process, and blameless reviews whose actions change systems and are tracked to completion. A principal engineer's particular contribution is to look across many incident reviews for patterns that no single team can see, such as a recurring lack of dependency documentation or a common absence of retrieval checks, and turn those into organization-wide improvements.

PART IV. INFLUENCE AND PEOPLE

8. Influence without authority and organizational dynamics

> *Senior ICs rarely have the power to order anyone to do anything. Their influence comes from credibility, relationships, evidence, and a working understanding of how their organization makes decisions. Engaging with organizational politics openly and constructively is part of the job rather than a distraction from it.*

The problem it addresses

A principal engineer may see clearly what the organization should do and still be unable to make it happen, because the teams that would need to change report to other managers, have their own priorities, and have no obligation to listen. Many technically excellent engineers respond by withdrawing ("I proposed it, they ignored it") or by escalating to leadership, which tends to work once and damage trust permanently.

The idea, explained simply

Influence without authority rests on a few reliable sources. The first is credibility, built through a track record of being right, being honest about uncertainty, and delivering what you said you would. The second is relationships, built before you need them, through helping others with their problems. The third is evidence: a prototype, a measurement, or a small pilot that shows an idea works reduces the perceived risk of adopting it far more than an argument does. The fourth is understanding what each stakeholder cares about and framing proposals in those terms, a theme going back to Cohen and Bradford's *Influence Without Authority*, which describes influence as an exchange in which you offer what others value.

Pre-wiring is a practical technique: before a decision meeting, talk with each important stakeholder individually, hear their concerns, and adjust the proposal. By the time of the meeting, there should be no surprises. Goedecke argues, in his writing on politics as a staff engineer, that engineers can and should take part in organizational politics without scheming. In his framing, this means understanding what leadership is trying to achieve, aligning technical proposals with those goals, and building a reputation as someone leaders can trust to deliver what matters to them. He also argues that in large companies most important changes require coordinating many people, so the ability to do this is a core engineering skill rather than an optional social one.

Partnership with engineering managers and product managers is a special case. The most effective senior ICs treat their manager and their product partner as collaborators with different responsibilities, agree explicitly on who owns which decisions, and present a united front to their teams.

EXAMPLE

A principal engineer believes every product team should adopt a shared tracing format for agent runs so that quality can be compared across products. Two of five teams are enthusiastic; one is

indifferent; two are opposed because they are under deadline pressure. Rather than escalating, the engineer builds an adapter that converts one opposed team's existing logs into the shared format with a single line of configuration, then uses the result to find a bug in that team's agent that had been causing silent failures. The team lead, now seeing direct value, agrees to adopt the format. The engineer then pre-wires the proposal with each remaining manager before the architecture review, framing it for one in terms of debugging time and for another in terms of the upcoming compliance audit. At the review, the proposal passes with little debate.

Alternatives people use

Escalation to a shared manager is the most common alternative and has its place for genuine deadlocks, but used routinely it marks the engineer as someone who cannot work across teams. Mandates through leadership achieve compliance quickly but often produce resentment and minimal effort. Some engineers rely on being technically right and expect the argument to win on its merits, which works in organizations with very strong engineering cultures and rarely elsewhere. Others avoid politics altogether, which leaves the field to those who engage with it, not always for good ends.

What works best

The approach the sources favor is to build credibility and relationships continuously, to lead with evidence such as prototypes and measurements, to frame proposals in terms of what stakeholders care about, to pre-wire significant decisions, and to escalate rarely and transparently, with the other party's knowledge. An early-stage principal should expect that the first few cross-team initiatives will take longer than planned and should treat each as an investment in relationships that will make later ones easier.

9. Writing and communication

> *Writing is the primary tool of senior IC work because it scales across time, teams, and seniority levels in a way that conversation does not. Good senior writing puts the conclusion first, is shaped for its specific reader, and is short enough to be read.*

The problem it addresses

As your scope grows, more of your impact depends on people you rarely talk to: engineers in other teams, executives, new hires a year from now. Meetings do not reach them, and verbal agreements fade. Yet many engineers write documents that bury the point, assume context the reader does not have, or are so long that nobody finishes them.

The idea, explained simply

Several established practices recur in the sources. BLUF, or bottom line up front, means the first one or two sentences state the conclusion, recommendation, or request. Audience shaping means writing differently for different readers: an executive needs the decision required, the options, the recommendation, and the risk; an engineer implementing the change needs the details and edge cases. Structure helps readers skim: headings that state claims rather than topics, short paragraphs, and tables for comparisons. Amazon's narrative memo culture, in which meetings begin with silent reading of a carefully written document, is the most developed example of an organization built around writing.

Regular status writing is a distinct skill. A short, predictable update, sent on a fixed cadence and in a consistent format, builds trust because leaders learn they do not need to chase you. Good updates state progress honestly, surface risks early, and say clearly when a decision is needed from the reader.

Spoken communication matters too. The senior IC is often the person who explains a technical situation to non-technical leaders, and the skill there is the same: start with what they need to know, use their vocabulary, and be explicit about uncertainty.

EXAMPLE

A weak executive update reads: "This week we continued work on the evaluation framework. We integrated the LLM judge and ran several experiments with different rubrics. Some results were promising, though there were issues with consistency. We also discussed the model upgrade."

A strong version of the same update reads: "We recommend delaying the model upgrade by two weeks. Our new evaluation shows the upgraded model is better on most tasks but worse on refund requests, which are about fifteen percent of traffic; shipping now would likely increase escalations to human agents. We have a prompt fix in testing and expect a clear answer by the twenty-first. No decision is needed from you unless you want to ship sooner and accept that risk." The second version is shorter, states the recommendation first, quantifies the risk, and makes clear what is and is not being asked.

Alternatives people use

Many engineers rely on meetings and conversation, which feel faster but do not scale and leave no record. Slide decks are common for leadership communication; they are good for presenting but poor for reasoning, because bullet points allow vague logic to look complete. Some engineers write extensive, comprehensive documents in the belief that completeness signals rigor, which often means nobody reads them.

What works best

Write more, write shorter, and write for a specific reader. Put the conclusion first. Use a consistent update format on a predictable cadence. For important documents, ask one person from the target audience to read a draft and tell you what they think it says; the gap between that and what you intended is the revision you need. For an AI engineer, a particularly valuable habit is explaining probabilistic results clearly to non-specialists: what was measured, on what data, how confident you are, and what it means for the decision.

10. Multiplying others: glue work, mentoring, sponsorship

> *A large share of a senior IC's impact comes through other people. That includes the coordinating and connecting work that holds teams together, mentoring that helps others improve, and sponsorship that creates opportunities for them. These are part of the job at staff and principal levels and should be done visibly and deliberately.*

The problem it addresses

The work that makes teams effective, such as onboarding new people, noticing that two teams are building the same thing, reviewing designs, improving processes, and keeping a project's loose ends tied, is often invisible. Reilly's "Being Glue" describes how engineers who do this work early in their careers can be passed over for promotion because they did not ship enough code, even though the team depended on them. At the same time, senior engineers who neglect this work cap their own impact at what they can personally produce.

The idea, explained simply

Reilly's argument has two parts. First, glue work is expected and valuable at senior levels, so senior engineers should do it deliberately and make it visible, for example by describing its outcomes in their brag document and in conversations with their manager. Second, it is risky for less senior engineers to do it unrecognized, so senior engineers and managers should notice who is doing it and make sure it is valued and shared fairly.

Mentoring is direct help with someone's growth: reviewing their designs carefully, explaining your reasoning, giving honest feedback, and helping them choose work that stretches them. The most effective mentoring is often embedded in ordinary work, such as pairing on a hard problem or explaining a design trade-off in a review comment, rather than in scheduled sessions.

Sponsorship is different and more powerful. It means using your reputation and access on someone else's behalf: recommending them to lead a project, naming their work in a leadership meeting, or giving them the visible part of a project you could have kept. Lara Hogan and others have argued that sponsorship is what most changes the trajectory of careers, and that it is particularly important for people who are less likely to be noticed otherwise.

EXAMPLE

A principal engineer notices that a mid-level engineer has, without being asked, become the person who keeps the evaluation datasets clean and who answers every other team's questions about them. This work is critical but appears nowhere in that engineer's goals. The principal does three things. They talk with the engineer's manager to make sure the work is recognized and becomes part of their formal responsibilities. They suggest that the engineer write a short internal guide on dataset hygiene, which the principal reviews and then shares widely with the engineer's name prominently on it. And when leadership asks who should lead a new initiative on evaluation data quality, the principal recommends the engineer rather than taking it on themselves, and

offers to advise. A year later, that engineer is promoted to senior with a clear record of cross-team impact.

Alternatives people use

Some senior engineers avoid glue work entirely and focus on technical output, which protects their own time but leaves their teams less effective. Others do all of it themselves, becoming bottlenecks and preventing others from growing. Formal mentoring programs are common and helpful, but tend to produce advice rather than opportunity. Many people mentor generously but sponsor rarely, because sponsorship involves risk to one's own reputation.

What works best

Do glue work deliberately and make its outcomes visible. Notice who else is doing it and make sure they are recognized. Mentor through ordinary work. Sponsor actively, which includes giving away visible opportunities you could have kept. For an early-stage principal, a useful habit is to ask each quarter which two or three people you have helped grow, and what you did for them that they could not have gotten elsewhere.

PART V. CRAFT

11. Staying technical at a senior level

> *Senior ICs must stay close enough to real technical work to keep their judgment sound, without taking on implementation that blocks others or that a less senior engineer could do. The usual resolution is to choose technical work that is off the critical path, that builds understanding, or that only you can do.*

The problem it addresses

As scope grows, the time available for hands-on technical work shrinks. Engineers who stop building entirely gradually lose touch with how systems actually behave, and their advice becomes abstract and less trusted. Engineers who keep building as much as before take on critical-path work they cannot finish because they are constantly pulled into meetings, and become a bottleneck for their own teams.

The idea, explained simply

The sources suggest choosing technical work by its properties rather than its interest. Good technical work for a principal has one or more of these properties. It is off the critical path, so that if you are pulled away, nothing is blocked. It builds understanding of the system, such as reading traces, debugging a production issue, or building a prototype to test an idea. It sets an example or a standard, such as a reference implementation that other teams will copy. Or it requires skills or context that only you have, such as the first version of a tricky cross-system integration.

Code review is a particularly high-leverage form of staying technical, because it keeps you aware of how the system is changing and lets you teach through specific feedback. Goedecke stresses that in large, established codebases, consistency with existing patterns is usually more valuable than local elegance, and that senior engineers should model this rather than introducing their own preferred style. Reading production data, whether logs, metrics, or for AI systems the traces of actual model runs, is another form of technical work that pays back directly in judgment.

EXAMPLE

A principal AI engineer sets aside two mornings a week for technical work. In a typical month, they build a prototype of a new routing approach that sends simple requests to a smaller model, to test whether the cost saving justifies the quality loss on the team's eval set. They spend several hours reading fifty production traces from a struggling agent, which produces more useful insight than any dashboard. They review a handful of significant pull requests carefully, writing comments that explain the reasoning. They do not take on the team's sprint commitments, because they know their schedule is too unpredictable. When the routing prototype shows a strong result, they write a short design document and hand the production implementation to a senior engineer on the team.

Alternatives people use

Some principals stop writing code entirely and focus on architecture and strategy. This can work in very large organizations with deep layers of staff engineers beneath them, but it often leads to a gradual loss of credibility. Others continue to own large implementation tasks, which keeps them sharp but makes them a bottleneck. Some pursue side projects unrelated to their teams' work, which keeps skills current but does not improve their understanding of the systems they advise on.

What works best

Protect regular time for technical work, choose work that is off the critical path and builds understanding, review code thoughtfully, and read production data directly. For AI engineers, reading traces is perhaps the single most important technical habit to keep, because the behavior of probabilistic systems cannot be inferred from their code.

12. Technical judgment and taste

> *Judgment is the ability to choose well among options when the right answer is not obvious. For senior engineers, it shows up most often in preferring simple, well-understood solutions, in spending novelty only where it creates advantage, in respecting the reasons behind existing systems, and in being honest about uncertainty.*

The problem it addresses

Many technical failures come not from lack of skill but from poor choices: adopting a fashionable technology that the team cannot operate, building an elaborate abstraction for a problem that turned out to be simple, or rewriting a system that only needed small changes. These choices are especially tempting in AI engineering, where new frameworks, techniques, and models appear constantly.

The idea, explained simply

A few well-known ideas capture much of what the sources mean by good judgment. McKinley's "Choose Boring Technology" argues that each team has only a few innovation tokens to spend on new technology, because new technology has unknown failure modes and operational costs. Spend them where they create real advantage, and use well-understood tools everywhere else. Hyrum's Law reminds you that every observable behavior of a widely used system will be depended upon by someone, so changes are always more costly than they look. Amazon's "respect what came before" tenet asks you to assume that existing systems had reasons before deciding they were mistakes. And Anthropic's guidance on building agents, which recommends starting with the simplest solution and adding complexity only when it measurably improves outcomes, is the same principle applied to AI.

Judgment also includes honesty about what you do not know. Strong senior engineers state their confidence explicitly, distinguish what they have measured from what they believe, and change their minds when evidence changes. Goedecke has written that good engineers are right a lot, but the sources are clear that this comes from careful reasoning and evidence rather than from projecting certainty.

EXAMPLE

A team proposes adopting a new multi-agent orchestration framework for a document-processing system, citing its flexibility. The principal engineer asks what problem the framework solves that the team's current approach does not. On examination, the current system is a sequence of four model calls with simple checks in between, which is a workflow in Anthropic's terms. The framework would add a dependency the team does not understand, hide the prompts behind abstractions, and make debugging harder, for flexibility the team does not yet need. The principal suggests keeping the current approach, adding evals to find which step is weakest, and revisiting the framework only if a concrete need for dynamic orchestration appears. Six months later, the system is still a workflow, and its quality has improved substantially through targeted prompt and retrieval changes.

Alternatives people use

Some engineers follow technology trends closely and adopt new tools early, which can create advantage but often creates operational burden. Others resist all change and keep systems on aging technology long after the costs exceed the benefits. Some rely on industry best-practice lists, which are useful starting points but do not account for local context.

What works best

Prefer the simplest solution that meets the need, spend novelty deliberately, understand existing systems before changing them, and be explicit about uncertainty. Judgment is learned mostly through feedback: make predictions, write them down, and check them later. Many senior engineers keep a decision journal in which they record significant choices, their expected outcomes, and what actually happened.

PART VI. THE AI ENGINEERING LAYER

13. Evaluation as the central discipline

> *For AI systems, evaluation is what testing is for conventional software, and more: it is how you know whether the system works, whether a change helped, and what to work on next. The current best practice is to start from error analysis of real traces, build a small number of targeted evals from what you find, validate any model-based graders against human judgment, and run evals continuously as part of development.*

The problem it addresses

Language model outputs vary from run to run and are hard to judge automatically. Without systematic evaluation, teams improve AI systems by intuition: someone tries a prompt change, looks at a few examples, and decides it seems better. This approach cannot detect regressions, cannot compare models, and cannot tell whether a fix for one problem has created another. It also leaves the most important question, what is actually going wrong for users, unanswered.

The idea, explained simply

Husain and Shankar's central recommendation is to start by looking at data. Take a sample of real traces, read each one fully, and write a short note about anything that went wrong (open coding). Then group those notes into a handful of failure categories (axial coding) and count how often each occurs. This tells you which failures matter most, and it often reveals problems nobody had anticipated. Only then decide which evals to build. Teams that skip this step tend to build evaluators for hypothetical failures and miss the real ones. A useful rule is to keep reading until new traces stop teaching you anything new.

Anthropic's guidance on agent evaluation adds structure. A task is a single test with defined inputs and success criteria; a trial is one attempt; a grader scores some aspect of the result. Graders come in three kinds. Code-based graders, such as checking that the right tool was called or that a database record exists, are fast, cheap, and reproducible but brittle. Model-based graders, such as an LLM judge applying a rubric, handle open-ended output but must be calibrated against human judgment and should be given an "unknown" option to reduce invented scores. Human graders are the gold standard and are mainly used to calibrate the other two. Anthropic recommends preferring code-based graders wherever they work.

Two further distinctions matter. Capability evals measure what the system cannot yet do and should start with a low pass rate, giving the team a target. Regression evals check what already works and should stay near a hundred percent. And because outputs vary, results across several trials should be summarized appropriately: pass@k when one success in k attempts is enough, pass^k when every attempt must succeed, as for a customer-facing agent. Anthropic notes that a seventy-five percent per-trial success rate implies only about a forty-two percent chance of three successes in a row, which is a sobering number for consistency-sensitive products.

Finally, eval-driven development means writing evals for a capability before building it, in the same way test-driven development writes tests first. This forces clarity about what success means and exposes vague product requirements early. Anthropic recommends starting with roughly twenty to fifty tasks drawn from real failures rather than waiting to build a large suite.

EXAMPLE

A team's internal IT helpdesk agent receives mixed feedback. The principal engineer pulls one hundred recent conversations and reads them with the team over two afternoons, writing a one-line note on each failure. Grouping the notes produces five categories: the agent asks for information the user already gave (thirty-one cases), it calls the password-reset tool for account-lockout problems (eighteen), it gives outdated instructions from an old knowledge article (fourteen), it is overly verbose (twelve), and miscellaneous (nine).

The team builds evals in proportion. The repeated-question failure gets an LLM judge with a narrow rubric, validated on forty hand-labeled examples until it agrees with human labels about ninety percent of the time. The wrong-tool failure gets a code-based grader that checks which tool was called on a set of thirty lockout scenarios. The outdated-instructions failure is traced to the knowledge base and fixed at the source, with a regression test. Verbosity is deferred because it matters least. Every prompt or model change now runs against these evals in CI, and the team repeats the trace-reading exercise monthly to find new failure categories.

Alternatives people use

The most common alternative is informal inspection, sometimes called "vibe checking," in which someone tries a few examples after each change. It is fast but cannot detect regressions or measure improvement. A second alternative is to adopt generic metrics from an evaluation library, such as off-the-shelf helpfulness or relevance scores, without first understanding the application's specific failures. These metrics are easy to compute but often do not correlate with what users care about. A third is to rely on public benchmarks, which are useful for comparing models in general but say little about a particular application. A fourth is to rely entirely on production metrics such as user ratings, which are valuable but arrive late and are noisy.

What works best

Start from error analysis of real traces, build targeted evals for the most frequent and most harmful failures, prefer code-based graders, validate any LLM judge against human labels before trusting it, separate capability evals from regression evals, report results across multiple trials with the appropriate metric, and run evals automatically whenever prompts, models, or tools change. Combine offline evals with online monitoring of production traffic and periodic human review. A principal AI engineer's distinctive contribution is often to make this practice standard across teams, providing shared tooling and templates, and above all to model the habit of reading traces personally.

14. Architecture judgment for agentic systems

> *The most reliable guidance on building AI agents is to use the simplest architecture that meets the need, to prefer predefined workflows over autonomous agents unless the problem genuinely requires flexibility, to design tools with the same care as human-facing interfaces, and to understand any framework you adopt well enough to see what it does underneath.*

The problem it addresses

The space of possible agent architectures is large and changes rapidly. Teams face pressure to adopt the newest multi-agent framework or the most autonomous design, often before they understand what their problem needs. Complex architectures are harder to debug, harder to evaluate, more expensive, and more prone to compounding errors, and their failures are harder to explain to stakeholders.

The idea, explained simply

Anthropic's "Building Effective Agents" draws a distinction that has become standard. In a workflow, the steps are defined in code and the model performs each one. In an agent, the model decides which steps to take and which tools to use, in a loop, until it judges the task complete. Workflows are more predictable and easier to test; agents are more flexible but cost more and can compound errors. The guidance is to start with a single well-designed model call, move to a workflow when a task naturally decomposes into fixed steps, and use an autonomous agent only when the number and nature of steps cannot be predicted in advance.

The same guidance names several common workflow patterns. Prompt chaining breaks a task into sequential steps, with checks between them. Routing classifies an input and sends it to a specialized handler, for example sending easy questions to a smaller model. Parallelization runs independent subtasks at once, or runs the same task several times and combines the results. Orchestrator-workers uses a central model to break a task into subtasks dynamically and delegate them. Evaluator-optimizer pairs a generating model with a critiquing model in a loop.

Tool design deserves particular attention. Anthropic calls it the agent-computer interface and argues that it deserves the same investment as a user interface: clear names, unambiguous parameters, good documentation in the tool description, and formats that are hard to misuse. In one example from their work on coding agents, simply requiring absolute rather than relative file paths eliminated a whole class of errors. Finally, the guidance on frameworks is cautious: they speed up getting started, but their abstractions can hide prompts and responses and encourage unnecessary complexity, so it is often better to start with direct API calls and adopt a framework only once you understand what it would do for you.

EXAMPLE

A team must build a system that reads incoming vendor invoices, extracts fields, checks them against purchase orders, and flags discrepancies. An early design uses a fully autonomous agent with access to a dozen tools. The principal engineer suggests instead a workflow: one call to extract fields into a structured schema, a code step to look up the purchase order, a second call to

compare and explain discrepancies, and a routing step that sends ambiguous cases to a human. Each step can be evaluated separately, failures are easy to attribute, and costs are predictable.

Later, a genuinely open-ended need appears: investigating unusual invoices that may require searching several systems in an unpredictable order. For that sub-problem alone, the team builds a small agent with four carefully designed tools, each with clear descriptions and constrained parameters, and an evaluation set of thirty historical investigations. The overall system remains a workflow, with one agentic step where autonomy earns its cost.

Alternatives people use

Some teams start with the most autonomous design available, reasoning that models will only get better. This can work for exploratory products but tends to produce systems that are hard to make reliable. Others adopt a comprehensive framework from the start to benefit from its features, at the cost of understanding. Some build multi-agent systems, with several specialized agents talking to each other, which can be valuable for complex tasks such as research but adds coordination overhead and new failure modes.

What works best

Use the simplest architecture that meets the need, measured by evals rather than by intuition. Prefer workflows where steps are predictable, and reserve autonomy for genuinely open-ended sub-problems. Invest heavily in tool design. Understand any framework before relying on it. Separate the probabilistic parts of the system, where the model makes judgments, from the deterministic parts, such as state changes and side effects, and make tools that change state safe to retry. Revisit architecture choices as models improve, since a task that needed a workflow last year may be handled by a single call today.

15. Context engineering

> *The quality of a model's output depends heavily on what is in its context window at each step. Context engineering is the practice of deciding what goes in, what stays out, and how it changes over the course of a task. The guiding principle is to provide the smallest set of high-signal information that lets the model do the task well.*

The problem it addresses

Early prompt engineering focused on writing one good static prompt. Modern agents run for many steps, call tools that return large results, accumulate long histories, and draw on external knowledge. Their context grows quickly, and as it grows, two problems appear. The model's attention to any particular piece of information degrades, so important instructions or facts get lost. And long-running tasks eventually exceed the context window altogether. Many agent failures that look like reasoning failures are actually context failures: the model did not have the right information, or had it buried among irrelevant material.

The idea, explained simply

Anthropic's writing on context engineering treats context as a finite resource with diminishing returns. Several practices follow. System prompts should be clear and specific at the right level of abstraction, neither so rigid that they hard-code brittle rules nor so vague that they give the model no guidance. Tools should return concise, relevant results rather than raw dumps. Instead of loading everything that might be relevant up front, agents can keep lightweight references, such as file paths or document identifiers, and retrieve details just in time when they are needed. For long tasks, three techniques help: compaction, which summarizes history as the context fills and continues from the summary; structured note-taking, in which the agent writes important state to an external file and reads it back later; and sub-agents, which take on a focused sub-task in a clean context and return only a condensed result.

Retrieval-augmented generation is one important case of context engineering. Its quality depends less on the choice of vector database than on chunking, ranking, and especially on checking whether what was retrieved is actually relevant before the model relies on it.

EXAMPLE

A research agent that compiles reports from internal documents performs well on short tasks but degrades on long ones, repeating earlier searches and forgetting the user's constraints. Reading traces shows that by step thirty the context contains tens of thousands of tokens of raw search results, most irrelevant, and the user's original instructions are far back in the history.

The team makes four changes. Search tools return short summaries with document identifiers, and a separate tool fetches full text when the agent asks for it. The agent maintains a running notes file of findings and open questions, which is placed near the end of the context at each step. When the context passes a threshold, older history is compacted into a summary that explicitly preserves the user's constraints. Deep dives into individual documents are delegated to a sub-

agent that returns a paragraph. Evals on long tasks improve substantially, and token cost per task falls by more than half.

Alternatives people use

The most common alternative is to rely on ever-larger context windows and include everything, which is simple but degrades quality and increases cost. Another is to focus exclusively on prompt wording, iterating on phrasing while ignoring what else is in the context. A third is to build elaborate retrieval infrastructure up front, before understanding what information the model actually needs.

What works best

Treat context as a scarce resource. Design tools to return concise, relevant output. Prefer just-in-time retrieval over loading everything. For long tasks, use compaction, external notes, and sub-agents. Diagnose context problems by reading traces and looking at what the model actually saw at the moment it went wrong. A principal engineer can add significant value by establishing shared context-management components and by teaching teams to ask "what did the model see?" before asking "why did the model reason badly?"

16. Running AI systems in production

> *Production AI systems need observability that captures full traces, online evaluation that detects quality changes, explicit management of cost and latency, guardrails around risky inputs and actions, and a disciplined process for changing models and prompts.*

The problem it addresses

AI systems that work well in development often behave differently in production. Users ask unexpected questions, model providers update models, data drifts, and costs grow with usage. Because failures are often quiet, such as a plausible but wrong answer, conventional monitoring based on errors and latency misses many of them. And because model and prompt changes alter behavior in ways that are hard to predict, they carry more risk than typical code changes.

The idea, explained simply

The foundation is tracing: recording, for every request, the full sequence of model calls, tool calls, inputs, outputs, latencies, and costs, in a form that can be searched and replayed. Without traces, error analysis and debugging are impossible. On top of tracing, online evaluation applies graders to a sample of production traffic continuously, so that quality changes are detected within hours rather than through customer complaints. Cost and latency are managed explicitly, through routing simpler requests to smaller models, caching repeated context, and setting budgets per request.

Guardrails sit around model calls to block or escalate unsafe inputs and outputs, and are particularly important for agents that take actions. A common principle is that the more consequential an action, the more it should be constrained: read-only tools can be used freely, while tools that send messages, change records, or spend money may require confirmation, narrow parameters, or human approval. Model and prompt changes are treated like deployments: they run through the eval suite, roll out gradually, and can be rolled back.

EXAMPLE

A company's sales assistant is about to switch to a newer model version. The principal engineer insists on the standard change process. The new model runs against the offline eval suite, where it scores higher overall but lower on one category: questions about contract terms, where it more often paraphrases instead of quoting. A prompt adjustment fixes this. The change then rolls out to five percent of traffic, with online graders comparing groundedness and task completion between old and new versions. After three days with no regression and a measurable improvement in completion, it rolls out fully. Cost per request rises slightly, so the team introduces routing that sends simple lookups to a smaller model, which brings costs back below the previous level with no measurable quality loss.

Alternatives people use

Some teams log only final inputs and outputs, which saves storage but makes debugging agents nearly impossible. Others rely on user feedback buttons as their only quality signal, which is useful but sparse and biased toward strongly negative or positive experiences. Some switch models as soon as new

versions appear, on the assumption that newer is better, which often introduces regressions in specific categories.

What works best

Full tracing as a non-negotiable foundation, online evaluation on sampled traffic, explicit cost and latency budgets, guardrails proportional to the consequences of actions, and a disciplined change process for models and prompts that includes offline evals, gradual rollout, and rollback. A principal AI engineer is often well placed to establish these as shared infrastructure so that each product team does not have to build them separately.

17. Working with coding agents

> *Coding agents amplify engineering expertise. Senior engineers get more out of them than juniors because the skills that make agent-assisted work reliable, such as writing clear specifications, decomposing problems, testing, and reviewing code critically, are senior skills. The best practice is to treat agents as fast but fallible collaborators within a strong engineering process.*

The problem it addresses

Coding agents can now write substantial amounts of working code from a description. This changes the economics of software work: code is cheap to produce, but understanding, verifying, and maintaining it is not. Used carelessly, agents produce large volumes of plausible code that nobody fully understands, with subtle bugs and inconsistent patterns. Used well, they let a single experienced engineer explore many more options, prototype faster, and spend more time on design and verification.

The idea, explained simply

Simon Willison's writing on what he first called vibe engineering, and now calls agentic engineering, makes the central argument: AI tools amplify existing expertise, so the more experience you have, the better the results. He notes that many practices recommended for working with agents, such as writing tests first, planning before coding, reviewing every change, and keeping good documentation, are things senior engineers already do. The practical consequence is that the senior engineer's role shifts toward specifying, verifying, and integrating.

Practices that recur in this literature include giving the agent clear context about the codebase in a persistent instructions file; asking it to plan before it implements and reviewing the plan; having it write or run tests so that its work is checked automatically; working in small, reviewable increments; reviewing generated code with the same rigor as human code, or more; and using agents freely for exploration and prototypes while being stricter about what reaches production. For a principal engineer specifically, agents make it much cheaper to build prototypes that demonstrate an idea, which, as chapter 8 discusses, is one of the most effective ways to influence decisions.

EXAMPLE

A principal engineer wants to show that a proposed caching approach would reduce agent costs across the organization. Without a coding agent, building a convincing prototype would take two weeks they do not have. Instead, they write a one-page specification describing the caching behavior, the interfaces, and the measurements needed, and ask a coding agent to plan the implementation. They correct the plan in two places where it misunderstood the existing code, then let the agent implement it in steps, running the existing test suite and a new benchmark after each one. They review the final diff carefully, find and fix one subtle cache-invalidation bug the tests did not catch, and add a test for it. In two days, they have a working prototype with measured cost savings on replayed production traffic, which they use to persuade three teams to adopt the approach. The production implementation is then done by those teams, with the prototype as a reference.

Alternatives people use

Some senior engineers avoid coding agents, often because of early experiences with poor output; this forgoes a large productivity gain. Some use them only for autocomplete or small snippets, which captures some benefit. At the other extreme, some accept agent output with minimal review, which is fast in the short term and expensive in maintenance and incidents.

What works best

Use coding agents extensively, with a disciplined process: clear specifications and persistent project context, planning before implementation, tests as automatic verification, small increments, and thorough review. Use them especially for prototypes, exploration, and the tedious parts of large changes. Keep ownership of design and correctness yourself. For a principal AI engineer, there is an additional benefit: working with coding agents daily builds direct intuition about agent behavior, tool design, and context management that transfers to the agentic systems you design for others.

PART VII. GROWTH

18. Deliberate growth and keeping up with the field

> *Improvement at senior levels does not happen through accumulated experience alone. It requires deliberate feedback, a record of your own work, active management of your time and energy, and, for AI engineers, a sustainable way of keeping up with a fast-moving field without being driven by hype.*

The problem it addresses

At junior levels, growth is guided by others: managers assign stretch work, reviewers point out mistakes, and promotion criteria are clear. At senior levels, this structure falls away. Feedback becomes rarer and less specific, because fewer people are positioned to judge your work. Your time is fragmented by requests. And in AI engineering, the field changes so quickly that it is easy either to fall behind or to spend all your time reading about new developments without applying any of them.

The idea, explained simply

Several practices recur in the sources. The first is seeking feedback deliberately: asking specific people specific questions, such as "what is one thing I could have done better in that design review?", rather than waiting for it. Reilly recommends building a network of peers at your level, inside and outside your company, who can give honest perspective. The second is keeping a brag document, a running record of what you did and what it achieved, which helps both with promotion and with seeing patterns in your own work. The third is managing finite time and energy; Reilly's chapter on finite time argues that senior engineers must decide explicitly what they will not do, because there will always be more good work than time. The fourth is choosing roles and projects for growth, including occasionally taking on work outside your comfort zone or moving between archetypes.

For keeping up with AI, the most effective practitioners combine a small number of high-quality sources, read regularly, with hands-on experiments. Reading model release notes and a handful of trusted engineering blogs is more useful than following every announcement. Running a small experiment on your own data, such as comparing two models on your eval set or trying a new context-management technique on a real agent, builds intuition that reading cannot. And being explicit with your organization about what is proven, what is promising, and what is hype is one of the most valuable services a principal AI engineer provides.

EXAMPLE

A principal AI engineer sets up a simple personal system. Each Friday, they spend fifteen minutes updating their brag document with what they did and its measured or observed effect. Once a month, they ask two people they worked closely with for one specific piece of feedback. They meet quarterly with a small group of principals from other companies to discuss hard problems. They block two hours a week for learning, alternating between reading and a small experiment, and keep a short running document of "things I tested and what I found," which they share with

their teams. When leadership asks whether a newly released model would help, they can usually answer within a day with results on the team's own eval set rather than with an opinion.

Alternatives people use

Many senior engineers rely on annual performance reviews for feedback, which is too infrequent and too general to guide improvement. Some try to keep up by following large volumes of social media and newsletters, which creates awareness but little depth and is easily dominated by hype. Others stop learning actively once they reach a senior level, relying on accumulated expertise, which works until the field shifts beneath them.

What works best

Seek specific feedback regularly, maintain a brag document, build a peer network, decide explicitly what you will not do, and combine a few trusted sources with hands-on experiments on your own data. Treat your own growth as a project with goals and periodic review, in the same way you would treat any other important piece of work.

19. A twelve-month plan for an early-stage principal AI engineer

> *This chapter turns the survey into a practical sequence. It assumes someone who has recently become a principal AI engineer, is technically strong, and wants to grow quickly into the full scope of the role.*

The problem it addresses

The practices in this survey are numerous, and trying to adopt them all at once leads to doing none of them well. A sequence helps, because some practices depend on others: you cannot set strategy without understanding the organization, and you cannot influence decisions without credibility.

The plan, explained simply

Months one and two: build the maps and agree the scope. Hold structured conversations with the people around your area, as described in chapter 3. Write down your locator, topographic, and treasure maps. Agree in writing with your manager what scope you are accountable for and which archetype the organization most needs from you now. Read a large sample of production traces from the most important AI systems in your scope, and write a short private assessment of where quality, cost, and reliability stand.

Months two to four: establish the evaluation foundation. Choose the system where better evaluation would matter most. Run an error analysis with that team, build targeted evals, validate any LLM judges, and wire the evals into the change process. Use this as the first visible demonstration of value. Start your brag document and your weekly update habit now.

Months four to six: write and land a first strategy. Based on what you have learned, write a strategy for one well-defined area, such as how the organization changes models and prompts safely, or how agents are traced and evaluated. Use the diagnosis, guiding policy, and coherent actions structure. Pre-wire it with each key stakeholder. Make sure it says what will not be done. Set up a regular review to keep it alive.

Months six to nine: lead a cross-team project to completion. Pick one project that implements part of the strategy across more than one team, such as adopting a shared evaluation or tracing platform. Lead it as DRI using the practices in chapter 6: a written definition of done tied to measured quality, a premortem, regular updates, early deployment behind flags. Finish it.

Months nine to twelve: multiply and reflect. Identify two or three engineers whose growth you can accelerate, and sponsor at least one of them for a visible opportunity. Hand off ownership of the systems you built so that they do not depend on you. Review your brag document, ask for feedback from several people, and revisit your scope and archetype with your manager. Update your strategy based on what you learned.

Throughout the year: protect time for technical work, especially trace reading and prototyping; use coding agents to make prototypes cheap; maintain a decision journal; and keep a small, regular practice of hands-on experiments with new models and techniques.

WHAT GOOD LOOKS LIKE AT THE END OF THE YEAR

At least one important AI system has a trustworthy evaluation suite that the team uses for every change. A written strategy exists for at least one area, people refer to it when making decisions, and it has been updated at least once. A cross-team project has shipped and is recognized by leadership as having achieved its goal. Several engineers can point to specific ways you helped them grow, and at least one has taken on a larger role with your sponsorship. Leadership asks for your view on AI technology decisions and expects an evidence-based answer. And you can describe, in your own brag document, a clear story of impact across a scope larger than any single team.

Alternatives people use

Some new principals start by writing an ambitious organization-wide vision in their first month, before they understand the terrain, which usually fails and costs credibility. Others spend the first year as an exceptional individual builder, which delivers value but delays the shift to principal-level impact. Some spend the year entirely in coordination and meetings, losing the technical credibility the role depends on.

What works best

Sequencing that builds understanding first, earns credibility through a concrete, visible improvement, then sets direction in a bounded area, then leads a cross-team effort to completion, and finally multiplies through others. Each step makes the next one easier. The plan should be adapted to the organization's real needs, but the sequence of understanding, demonstrating, directing, delivering, and multiplying holds up across most of the sources.

Summary table

The table below condenses each chapter into the problem it addresses, the most common alternative, and the practice the sources favor.

Area	Problem addressed	Common alternative	Favored practice
Role definition	Unclear expectations beyond senior	Senior engineer with more output	Accountability for a scope, leverage over personal output, technical depth as foundation
Choosing work	Calendar fills with low-impact work	Work from the request queue	Deliberate quarterly portfolio; avoid snacking and preening; favor neglected, important work
Situational awareness	Good ideas fail for organizational reasons	Rely on org chart or manager	Build locator, topographic, and treasure maps through structured listening
Strategy	Locally sensible choices add up to incoherence	Top-down vision or mandates	Diagnosis-first written strategy built from real decisions, with explicit trade-offs and a review mechanism
Decisions	Too slow on reversible choices, too fast on irreversible ones	Meetings without records; heavyweight boards	Proportional process by reversibility; design docs and ADRs; a named decider
Shipping	Projects stall in the last stretch	Rely on project managers or nobody	Single technical owner, definition of done, regular updates, early risk surfacing, incremental delivery
Reliability	Failures recur; quality degrades silently	No targets; blame; user complaints as monitoring	SLOs including quality; structured incidents; blameless reviews with systemic actions
Influence	No authority over the teams that must change	Escalation or mandates	Credibility, relationships, evidence and prototypes, stakeholder framing, pre-wiring
Writing	Impact depends on people you rarely talk to	Meetings and slide decks	Short, reader-specific documents with the conclusion first; predictable updates
Multiplying others	Essential work goes unrecognized; impact capped at one person	Avoid glue work or hoard it	Visible glue work, embedded mentoring, active sponsorship
Staying technical	Lose touch or become a bottleneck	Stop coding or keep owning sprint work	Protected time for off-critical-path work, reviews, trace reading, prototypes
Judgment	Poor technical choices despite skill	Follow trends or resist change	Simplest sufficient solution, deliberate novelty, respect for existing systems, explicit uncertainty
Evaluation	No way to know if AI changes help	Informal checks; generic metrics	Error analysis first, targeted evals, validated judges, capability and regression suites, continuous runs
Agent architecture	Unnecessary complexity and compounding errors	Most autonomous design; heavy frameworks	Simplest architecture; workflows by default; careful tool design; understand frameworks
Context engineering	Model lacks or loses the right information	Include everything; tune wording only	Minimal high-signal context, just-in-time retrieval, compaction, notes, sub-agents

Area	Problem addressed	Common alternative	Favored practice
Production AI	Quiet failures, cost growth, risky model changes	Log outputs only; switch models eagerly	Full tracing, online evals, cost and latency budgets, proportional guardrails, staged rollouts
Coding agents	Cheap code that nobody understands	Avoid agents or accept output unreviewed	Agents inside a disciplined process: specs, plans, tests, small increments, rigorous review
Growth	Feedback becomes rare; field moves fast	Annual reviews; high-volume news feeds	Specific feedback, brag document, peer network, few trusted sources plus experiments

Sources and further reading

Books

- Will Larson, *Staff Engineer: Leadership Beyond the Management Track* (2021). Companion site and stories at staffeng.com.
- Will Larson, *Crafting Engineering Strategy* (O'Reilly, 2025).
- Tanya Reilly, *The Staff Engineer's Path: A Guide for Individual Contributors Navigating Growth and Change* (O'Reilly, 2022).
- Richard Rumelt, *Good Strategy Bad Strategy* (2011).
- Allan R. Cohen and David L. Bradford, *Influence Without Authority*.
- Betsy Beyer et al., *Site Reliability Engineering* (Google, O'Reilly, 2016), especially the chapters on SLOs and postmortem culture.
- Chip Huyen, *AI Engineering* (O'Reilly, 2025).

Essays and articles

- Will Larson, "Work on What Matters," lethain.com/work-on-what-matters.
- Tanya Reilly, "Being Glue," noidea.dog/glue.
- Amazon, "Principal Engineering Community Tenets," amazon.jobs/en/landing_pages/pe-community-tenets.
- Sean Goedecke, "How I ship projects at big tech companies," seangoedecke.com/how-to-ship; "How I influence tech company politics as a staff software engineer," seangoedecke.com/how-to-influence-politics; and related posts on large codebases.
- Dan McKinley, "Choose Boring Technology," mcfunley.com/choose-boring-technology.
- Michael Nygard, "Documenting Architecture Decisions," cognitect.com/blog/2011/11/15/documenting-architecture-decisions.
- Julia Evans, "Get your work recognized: write a brag document," jvns.ca.
- Alex Ewerlof, "Staff archetypes are anti-patterns," blog.alexewerlof.com.
- Gergely Orosz, The Pragmatic Engineer newsletter, including interviews on staff-plus roles and shipping at large companies.

AI engineering

- Anthropic, "Building Effective Agents," anthropic.com/engineering/building-effective-agents.
- Anthropic, "Demystifying evals for AI agents," anthropic.com/engineering/demystifying-evals-for-ai-agents.
- Anthropic, "Effective context engineering for AI agents," anthropic.com/engineering/effective-context-engineering-for-ai-agents.
- Hamel Husain, writing on evals and error analysis, hamel.dev, and the AI Evals course with Shreya Shankar.
- Simon Willison, "Vibe engineering" (October 2025), simonwillison.net/2025/Oct/7/vibe-engineering, and the Agentic Engineering Patterns collection (2026).
- Eugene Yan, writing on applied LLM systems and evaluation, eugeneyan.com.

A note on sources: the claims attributed above to Larson, Reilly, Goedecke, Amazon, Anthropic, Husain, and Willison were checked against their published work or reliable summaries of it while preparing this survey. The examples are illustrative composites written for this document and do not describe specific companies.